# 05_Module_5_Incident_Management

# Module 5 — Incident Management & Response

## Duration: 8 Hours

---

## 5.1 The Incident Lifecycle

An incident is any event that disrupts or degrades a service users depend on. For a bank: failed transfers, ATMs offline, mobile app unresponsive — all incidents.

### The 5 Stages

```
1. DETECTION → 2. TRIAGE → 3. RESPONSE → 4. RESOLUTION → 5. REVIEW
```

### Stage 1 — Detection

| Source | Example | Time to Detect |
|---|---|---|
| Burn rate alert | Error budget burn rate > 2 for 2 min | Seconds |
| Monitoring dashboard | Engineer notices anomaly | Minutes |
| Customer complaint | User calls support | Minutes to hours |
| Regulatory notification | CBN queries an outage | Too late |

**Goal:** Detect via automated alerts before customers notice. If customers tell you about incidents before your monitoring does, your observability is broken.

### Stage 2 — Triage

Answer three questions: 1. How many users are affected? 2. Is this getting worse or stable? 3. What is the business impact right now?

**Severity levels:**

| Severity | Definition | Example | Response SLA |
|---|---|---|---|
| **SEV 1** | Complete service outage, all users | ATM network fully down | Immediate, any time |
| **SEV 2** | Major degradation, most users | 30% of mobile transfers failing | 15 min |
| **SEV 3** | Partial degradation, some users | Slow login for one region | 1 hour |
| **SEV 4** | Minor issue, few users | Error on rarely-used feature | Next business day |

### Stage 3 — Response

**Principle: mitigation before root cause.** Stop the bleeding first. Investigate after.

**Common mitigations:**

| Action | When |
|---|---|
| **Rollback** | Recent deployment caused the issue |
| **Restart** | Service is crashed or hung |
| **Failover** | Primary system is down, backup is ready |
| **Feature flag off** | One feature is broken, disable it |
| **Scale up** | System is overloaded, add capacity |
| **Block traffic** | Attack or bad input causing damage |

**Communication cadence:** Status update every 15 minutes for SEV 1, every 30 minutes for SEV 2.

```
[14:32] INCIDENT OPEN — SEV 2 — Mobile banking transfers failing
[14:35] On-call engineer engaged. Investigating logs.
[14:48] Root cause identified: database connection pool exhausted.
[14:52] Mitigation: increased connection pool from 50 to 200. Error rate dropping.
[15:01] Error rate back to normal. Monitoring 30 min before close.
[15:34] INCIDENT CLOSED. All services normal. Post-mortem scheduled.
```

### Stage 4 — Resolution

An incident is resolved when: - Service behaviour is normal - SLO is being met - No new errors being generated - Monitoring confirmed for at least 30 minutes

**Do not close early.** Premature closure is a common cause of re-occurrence.

### Stage 5 — Review (Post-Mortem)

The most important stage — and the one most teams skip. Covered in Section 5.3.

---

## 5.2 Incident Command System (ICS) for SRE

Adapted from emergency response (fire departments, FEMA). Clear roles prevent chaos under pressure.

### The Four Roles

| Role | Responsibility | Does NOT |
|---|---|---|
| **Incident Commander (IC)** | Runs the incident, makes decisions, delegates | Touch the keyboard |
| **Technical Lead** | Investigates cause, implements fix | Make strategic decisions |
| **Communications Lead** | Status updates, stakeholder mgmt | Debug |
| **Scribe** | Real-time documentation, timeline | Anything else — just writes |

### The IC Rule

> **The Incident Commander does not touch the keyboard.**

If the IC starts debugging, nobody is running the incident. The IC's job is to: - Assign tasks - Make prioritisation decisions - Ensure communication flows - Track the timeline - Declare when the incident is over

### ICS in Practice

```
You're the IC. You get: "ATM network is down."

What you do:
1. Confirm severity (SEV 1 — complete outage)
2. Assign Tech Lead to investigate
3. Assign Comms Lead to draft stakeholder notification
4. Set 15-min status update cadence
5. Ask: do we need to failover? Do we have a known mitigation?

What you DON'T do:
- SSH into a server
- Check Grafana
- Write code
```

---

# 5.3 Blameless Post-Mortems

### What is Blameless?

**Blameless does not mean no accountability.** It means: when investigating failures, we examine **systems, processes, and decisions** — not people.

**Why blameless works:**

| Blame-driven | Blameless |
|---|---|
| Engineers hide problems | Engineers report early |
| Post-mortems are defensive | Post-mortems are honest |
| Root cause is never found | Real cause is identified |
| Same incident repeats | Incident is prevented |
| Team culture erodes | Team gets stronger |

### The Key Insight

> "Every time something breaks, the correct question is: **what in our system allowed this to happen?** Not: who did this?"

Human error is never the root cause. The root cause is always a system that allowed human error to cause damage.

### Post-Mortem Structure

A good post-mortem has five sections:

**1. Summary** — What happened, what was the impact (2-3 sentences)

**2. Timeline** — Exact sequence of events with timestamps

**3. Root Cause + Contributing Factors** — Why it happened, what enabled it

4. **What Went Well / What Went Poorly** — Honest assessment of response

5. **Action Items** — Specific, owned, dated tasks to prevent recurrence

## Worked Example

```
## Incident Post-Mortem
**Date:** 2026-02-23 | **Severity:** SEV 2
**Duration:** 32 minutes (14:32 – 15:04)

---

## Summary
A marketing campaign launched at 14:00 caused a 3x traffic spike.
The database connection pool (max 50) was exhausted.
12,000 users affected, 847 transfers failed, ~NGN 2.3M impact.

## Timeline
| Time | Event |
|------|-------|
| 14:32 | Alert fires: MobileTransferErrorRate > 5% |
| 14:35 | On-call acknowledges |
| 14:38 | Incident opened SEV 2, IC assigned |
| 14:48 | Root cause: connection pool exhausted |
| 14:52 | Pool limit increased 50 → 200 |
| 14:55 | Error rate dropping |
| 15:01 | Error rate normal |
| 15:04 | Incident resolved |

## Root Cause
Connection pool limit of 50 was set during initial deployment 18 months ago.
Traffic grew 5x since then. No capacity review conducted.

## Contributing Factors
- No monitoring on connection pool utilisation
- No alert when pool exceeded 70%
- Marketing did not notify SRE of campaign
- No capacity review process existed

## What Went Well
- Alert fired within 3 min of incident start
- On-call responded quickly
- Root cause identified within 13 min
- Fix was straightforward

## What Went Poorly
- No runbook for connection pool exhaustion
- Comms lead not engaged until 20 min in
- Stakeholder updates were delayed

## Action Items
| Action | Owner | Due |
|--------|-------|-----|
| Increase pool limit to 200 + auto-scaling | DB team | Mar 1 |
| Add alert for pool utilisation > 70% | SRE team | Feb 28 |
| Write runbook for pool exhaustion | On-call engineer | Feb 28 |
| Create process: marketing notifies SRE of campaigns | EM | Mar 7 |
| Schedule quarterly capacity reviews | SRE Lead | Mar 1 |
```

---

# 5.4 Chaos Engineering

## What It Is

Chaos engineering is the practice of **deliberately introducing failures** in a controlled way to discover weaknesses before real users find them.

> "If your system can handle failures in testing, it will handle them in production."

## Origin

Netflix invented Chaos Monkey — software that randomly shut down production servers during business hours. The logic: if it can happen accidentally, let's make it happen on purpose when we're watching.

## Banking Chaos Experiments

| Experiment | What It Tests |
|---|---|
| Kill primary database | Does failover work? How long? Are transactions lost? |
| Add 500ms network latency | Does the app handle slow connections? Timeouts? |
| Fill disk to 95% | What happens to logs? Does the app degrade gracefully? |
| Kill one app instance | Does load balancing redirect traffic? |
| Simulate vendor API timeout | Does the system queue or fail? |
| Expire all auth tokens | Does re-auth work? Can users still access their accounts? |

## Game Day Structure

A Game Day is a scheduled, structured chaos experiment:

```
1. HYPOTHESIS — What do you expect to happen?
   "If we kill the primary database, failover completes in < 30s with no data loss."

2. BLAST RADIUS — Limit scope
   "Staging environment, Tuesday 10-11am."

3. RUN — Introduce the failure
   Kill primary database service.

4. OBSERVE — Watch what actually happens
   Measure failover time. Count any failed transactions.
   Did monitoring detect it? Did alerts fire?

5. DOCUMENT — Record what you learned
   "Failover: 47s. 3 in-flight transactions lost.
    SLO impact: 47s downtime.
    Monitoring detected it in 15s."

6. FIX — Address findings
   Improve failover speed. Add transaction retry logic.
   Run again in 2 weeks to verify fix.
```

**Simple chaos demo:**

```
# Simulate a service crash
kubectl delete pod transfer-service-abc123 -n production
```

```
# Watch Kubernetes restart it automatically
kubectl get pods -n production -w

# Check: how long was the service down?
# Did the burn rate alert fire?
# Was the SLO impacted?
```

---

# 5.5 Classroom Activity: Full Incident Simulation

**Time:** 1.5 hours

### Scenario: “The Friday Afternoon Disaster”

```
It is 4:00 PM on Friday 28 February 2026. Alert fires:

CRITICAL: ATM_NETWORK_DOWN — 0% success rate

All 47 Union Bank ATMs across Lagos are offline.
End-of-month salary payments are due.
Branches close in 1 hour.
Social media is already active with complaints.
```

**Part 1 — Role Play (30 min):**

Assign four roles: - **Incident Commander** — coordinates, does NOT debug - **Technical Lead** — investigates cause, implements fix - **Communications Lead** — 3 stakeholder updates during the incident - **Scribe** — documents everything

Use real monitoring data (provided) to investigate: - Grafana shows error rate spike at 15:45 - Kibana shows “network timeout” errors on core banking API - PagerDuty shows no prior alerts for core banking - Last deployment was this morning at 10:00

**Part 2 — Debrief (20 min):** - What decisions did the IC make? - Was communication clear? - What information did you wish you had? - When should you have escalated? - What would you add to the runbook?

**Part 3 — Post-Mortem (40 min):** Each group writes a post-mortem using the template. Include: - Timeline (realistic based on the scenario) - Root cause analysis (5 Whys) - Contributing factors - Action items (with owners and due dates)

---

# Key Terms

| Term | Definition |
|---|---|
| **Incident** | Any event that disrupts or degrades a service users depend on |
| **Incident Commander** | Runs the incident response — coordinates, does not debug |
| **Mitigation** | Stopping user impact before finding root cause |
| **Post-Mortem** | Written review focused on learning, not blame |
| **Blameless** | Investigating systems/processes, not individuals |
| **Chaos Engineering** | Deliberately introducing failures to discover weaknesses |
| **Game Day** | A structured chaos experiment with hypothesis, blast radius, and fix |
| **5 Whys** | Root cause analysis technique — ask “why” five times |
| **Blast Radius** | The scope of impact of a failure or experiment |

# Further Reading

- Google SRE Book, Chapter 14 — Managing Incidents
- Google SRE Book, Chapter 15 — Postmortem Culture
- Chaos Engineering Principles
- Incident Command System for IT — PagerDuty